

Database Tables —Detailed Guide

Updated: 2026-04-17 | Based on actual production DB (5 patients processed)
Total: 12 tables (9 application + 3 auth) | Branch: feat/save-intermediate-results

1. transcript_analysis_log

Pipeline execution record —stores the fact that “this patient file was processed”along with all metadata about the processing.

When the pipeline processes Input_Keystrokes_REC001 (SID 14) .xlsx, one row is created in this table. It records when processing started (pipeline_started_at), how many sentences were analyzed (total_sentences=424), what settings were used (top_n=10), a backup of the result xlsx (xlsx_data), whether the AI pipeline completed (processed=True), and the overall consultation quality score (ai_overall_score=3.67). Next time the same file is encountered, this table is checked and already-processed files are skipped.

Column	Type	Sample (SID_10)	Description
id	SERIAL PK	1	Unique analysis run ID
patient_id	VARCHAR	SID_10	Patient identifier (extracted from filename)
total_sentences	INT	428	Number of sentences after R stringi segmentation
top_n	INT	10	Top-K sentences selected per domain
context_window	INT	3	±3 surrounding sentences for context
model_results	JSONB	None	(Deprecated —kept for backward compatibility)
xlsx_data	BYTEA	(18,300 bytes)	Result xlsx backup. Fallback when disk file deleted
source_filename	VARCHAR	Input_Keystrokes_REC_001 (SID 10).xlsx	Original input filename
pipeline_started_at	TIMESTAMP	05:26:14	When pipeline_runner began processing
analyzed_at	TIMESTAMP	05:26:30	When NLP results saved to DB (Step 8)
ai_overall_score	FLOAT	3.4	Average of all domain ai_scores (0-5). Displayed on DOCTOR page
processed	BOOLEAN	True	True when full pipeline (NLP + AI) completed
processed_at	TIMESTAMP	05:29:28	When AI pipeline (Step 9) completed

Used by: - persistence.file_already_processed() —skip already-processed files - GET /api/transcript/download/{patient_id} —xlsx download fallback from xlsx_data - GET /api/transcript/history/{patient_id} —analysis run history - llm_domain_scoring_and_summary.analysis_id FK target

2. sentence_prediction

NLP model predictions —stores the Top-10 sentences per domain with their probability scores, as determined by the 5 NLP classification models.

For example, out of SID 10's 428 sentences, the cp (cancer prognosis) model assigned 95.1% probability to "so i'm going to take that 12 percent and cut it in half...", making it one of the Top-10 for that domain. 5 domains × 10 sentences = 50 rows per patient.

Where this appears in the UI: In PatientInitialVisitReportV35.tsx, each TopicCard has a collapsible "Evidence" section. When the patient clicks "Show Evidence", it displays a list of sentences ranked by pred_score. These sentences come from this table via GET /api/patient/sentences/{file} (Top-7 per domain).

Column	Type	Sample (SID_10, cp)	Description
id	SERIAL PK	1	Unique prediction ID
analysis_id	INT FK	1	→ transcript_analysis_log.id
patient_id	VARCHAR	SID_10	Patient identifier
model	VARCHAR	cp	Domain (cp/le/ed/inc/ius)
sentence_index	INT	166	Global sentence sequence number
utterance_index	INT	67	Original utterance number (i)
sentence_in_utterance	INT	3	Sentence number within utterance (i2)
speaker	VARCHAR	Interviewer:	Speaker
sentence_text	TEXT	"so i'm going to take that 12 percent and cut it in half..."	Original sentence
pred_score	FLOAT	0.951	NLP probability (95.1% related to cp)
context	TEXT	"it removes the testosterone...<main>so i'm going to...</main>..."	±3 surrounding sentences

Used by: - GET /api/patient/sentences/{file} —evidence sentences on PATIENT page - Analysis result reconstruction from DB

3. doctor_rewrite_log

Doctor rewrite history —records every time a doctor modifies a sentence on the dashboard, storing both the original and revised versions with timestamps.

Currently 0 rows because no doctor has rewritten any sentences yet. When a doctor changes "so i'm going to take that 12 percent..." to "Your cancer risk is about 6% with treatment", both versions are INSERT'd here. Multiple rewrites of the same sentence create multiple rows, preserving the full edit history.

Where this appears in the UI: In PhysicianReportsModifiedV41Timothy.tsx, the **GridView** has a "Suggested Rephrasing" column. When the doctor clicks "AI Rewrite", GPT-4o generates an improved version. If the doctor accepts and saves, the original and revised sentences are stored here. The **HistoryModal** component shows the full rewrite history for a selected sentence.

Column	Type	Example	Description
file	VARCHAR PK	Input_Keystrokes REC 001 (SID 10).xlsx	Target file
i	INT PK	67	Target utterance number
i2	INT PK	3	Target sentence number
time	TIMESTAMP PK	2026-04-17T14:30:00Z	Rewrite time (multiple rewrites = multiple rows)
speaker	VARCHAR	Interviewer:	Speaker
original_sentence	TEXT	"so i'm going to take..."	Original sentence
revised_sentence	TEXT	"Your cancer risk is about 6%..."	Doctor's rewritten version
score	FLOAT	4.0	Quality score after rewrite

Column	Type	Example	Description
class	VARCHAR	cancer_prognosis	Domain

Used by: - GET /api/doctor/rewrites —rewrite history on DOCTOR page - GET /api/doctor/rewrites/{file}/{i}/{i2}/history —per-sentence rewrite history

4. patient_summary

Patient summary parent record —represents “a summary exists for this patient”. Acts as the FK parent for patient_summary_domain (5 domain rows per patient).

One row per patient. The actual domain-level data (summary_text, patient_scoring) lives in the child table patient_summary_domain. This table exists because DB design requires a “1” side table in a 1:N relationship for FK integrity. Deleting a patient here CASCADE-deletes all 5 domain rows.

Column	Type	Sample	Description
file	VARCHAR PK	Input_Keystrokes REC 001 (SID 10).xlsx	Patient file
speaker	VARCHAR PK	Patient_Input_Keystrokes REC 001 (SID 10)	Patient identifier
entire_summary	TEXT	None	Full visit summary (currently unused)

Used by: - GET /api/patient/summaries/{file}/{speaker} —JOINS with patient_summary_domain - FK parent for patient_summary_domain CASCADE delete

5. patient_summary_domain

Patient per-domain feedback —stores the star rating and text response that the patient enters on the dashboard for each domain. The pipeline creates empty rows (NULL values), and the patient fills them in later during a follow-up visit.

Like a patient survey form: the pipeline creates 5 blank domain cards (cp, le, ed, inc, ius), and when the patient clicks a rating for “cancer prognosis”, patient_scoring is UPDATE’d from NULL to the rating value. Note: patient_response (free text) has an API but no UI implementation yet.

Where this appears in the UI: In PatientInitialVisitReportV35.tsx, each **TopicCard** has a NIH PROMIS unipolar helpfulness scale (5 levels: “Not at all helpful” to “Extremely helpful”) at the bottom. When the patient selects a level, updateSingleClassScore() is called → PUT /api/patient/scoring → this table’s patient_scoring column is UPDATE’d. The display_order determines the vertical order of the 5 topic cards on the page (1=Cancer Prognosis at top, 5=Life Expectancy at bottom).

Column	Type	Sample (SID_10, cp)	Description
file	VARCHAR PK,FK	Input_Keystrokes REC 001 (SID 10).xlsx	Patient file
speaker	VARCHAR PK,FK	Patient_...SID 10	Patient identifier

Column	Type	Sample (SID_10, cp)	Description
domain	VARCHAR PK	cancer_prognosis	Domain
display_order	INT	1	UI display order (1=cp, 2=inc, 3=ed, 4=ius, 5=le)
summary_text	TEXT	""	Domain summary text (currently empty)
patient_scoring	INT	None	Patient star rating 0-10 (NULL until patient rates)
patient_response	TEXT	None	Free text feedback (API exists but UI NOT implemented)

Used by: - GET /api/patient/summaries/{file}/{speaker} —domain summary cards on PATIENT page - PUT /api/patient/scoring —patient star rating (PatientInitialVisitReportV35.tsx) - PUT /api/patient/responses —patient text response (API only, UI not implemented)

6. survey_submission_log

Patient survey responses —stores the full JSON answers when a patient submits SDM, DCS, Risk Perception, or Satisfaction surveys on the follow-up page, and tracks synchronization status with the REDCap research database.

Currently 0 rows because no patient has submitted a survey yet. When a patient completes the DCS (16 questions), the entire response is stored as JSON in answers. Immediately after, a background task calls the REDCap API to sync. Same patient submitting the same survey again creates a NEW row (INSERT, not UPDATE) to preserve responses at different timepoints.

Where this appears in the UI: In PatientFollowUpReportV31Re.tsx, the follow-up visit has a multi-step survey flow: Welcome → SDM (9 questions) → DCS (16 questions) → Risk Perception (with collapsible AI summaries) → Patient Satisfaction → Completion. Each survey shows one question at a time. When the patient clicks “Submit”, submitSurvey() from surveyApi.tsx calls POST /api/surveys/submit, which INSERTs a row here. On page refresh, fetchSurveySubmissions() restores previous answers so the patient can continue where they left off.

Column	Type	Example	Description
id	SERIAL PK	1	Submission ID
file	VARCHAR FK	Input_Keystrokes REC001 (SID 14).xlsx	Patient file
speaker	VARCHAR FK	Patient_...SID 14	Patient identifier
survey_type	VARCHAR	dcs	Survey type (sdm/dcs/risk_perception/satisfaction)
answers	JSONB	{q1: 2, q2: 3, ..., q16: 4}	Full survey response
extra_data	JSONB	{browser: Chrome, session: abc}	Metadata
submitted_at	TIMESTAMP	2026-04-17T16:05:30Z	Submission time
redcap_synced	BOOLEAN	true	REDCap sync success
redcap_record_id	VARCHAR	REC-2026-0014	REDCap record ID (on success)
redcap_error	TEXT	None	Error message (on failure)

Used by: - POST /api/surveys/submit —survey submission from PATIENT follow-up page - GET /api/surveys/by-speaker/{speaker} —restore previous responses on page refresh - Background REDCap sync worker

7. llm_domain_scoring_and_summary

GPT-4o AI pipeline results—stores the per-domain evaluation of “how specifically did the doctor communicate risk to the patient”, including a 0-5 score, extracted numerical estimates, and a patient-friendly reformatted summary.

While NLP (sentence_prediction) determines “which sentences are related to this domain”, this table stores “how specifically did the doctor communicate risk in those sentences”. Example: “There’s some cancer risk”= ai_score 1 (vague), “24% risk without treatment, decreases to 6% with surgery”= ai_score 4 (very specific). The 33 rows (more than 5×5=25) occur because some domains have multiple treatment comparisons stored as separate rows.

Where this appears in the UI (two different pages, two different columns):

1. **Doctor page** (PhysicianReportsModifiedV41Timothy.tsx):
 - **DashboardView**: The patient list shows each patient’s overallScore (from ai_overall_score in transcript_analysis_log, which is the average of this table’s ai_score). Patients are color-coded: green (score ≥ 4), yellow (2-3), red (< 2).
 - **GridView**: The “Your Score” column shows per-domain ai_score (0-5) for the selected patient. The **DetailView** shows a Recharts LineChart trajectory of ai_score over time via GET /api/doctor/scores/trajectory.
2. **Patient page** (PatientInitialVisitReportV35.tsx and PatientFollowUpReportV31Re.tsx):
 - Each **TopicCard** displays the reformat_sentence as the main AI summary text. For example, the Cancer Prognosis card shows: “Your doctor noted that, without treatment, your risk of dying of cancer is 24-25%. With treatment, this risk decreases to 6%.” This text comes from GET /api/patient/ai-summary/{file} → this table’s reformat_sentence column.

Column	Type	Sample (SID_10, cp)	Description
id	SERIAL PK	1	Result ID
analysis_id	INT FK	1	→ transcript_analysis_log.id
patient_id	VARCHAR	SID_10	Patient identifier
domain	VARCHAR	cp	Domain
ai_score	INT	4	GPT-4o score 0-5 → DOCTOR page
score_explanation	TEXT	“Does the text mention cancer mortality?...score is 4.”	GPT-4o chain-of-thought reasoning (from ai_pipeline/scoring.py)
extracted_estimate	TEXT	“24, 25 percent—down to six percent”	Extracted risk numbers
treatment	VARCHAR	None	Related treatment (ed/inc/ius only)
source_sentence	TEXT	“so i’m going to take that 12 percent...”	AI-selected original sentence
source_context	TEXT	“it removes the testosterone...<main>...</main>...”	Surrounding context
reformat_sentence	TEXT	“Your doctor noted that your risk of dying of cancer is 24-25%...”	Patient-friendly text → PATIENT page
source_filename	VARCHAR	Input_Keystrokes REC 001 (SID 10).xlsx	Source file
created_at	TIMESTAMP	05:29:28	Creation time

Used by: - GET /api/doctor/scores/average —consultation quality score on DOCTOR page - GET /api/doctor/scores/trajectory —score trend over time on DOCTOR page - GET /api/patient/ai-summary/{file} —AI summary card on PATIENT page - POST /api/doctor/ai-rewrite —AI rewrite suggestions using source_sentence + source_context

8. user_interaction_log

User behavior tracking —records every user action on the dashboard (clicks, scrolls, mouse movements, dwell time) in real-time for research analysis of how patients and doctors interact with the consultation information.

For example, when a patient hovers near the “cancer prognosis” card for 38 seconds, an event is recorded with `event_type=cursor_proximity_leave` and `hoverDuration: 38007071ms`. This data enables research questions like “Which domain information do patients spend the most time reading?” or “Which sentences do doctors click on most?”

Where this appears in the UI: This table is NOT visible to patients or doctors. It is consumed by the **AdminTrackingDashboard** (`AdminTrackingDashboard.tsx`), which shows event counts, session timelines, per-patient behavior heatmaps, and event type distributions. Accessible only to administrators.

Column	Type	Sample	Description
<code>id</code>	SERIAL PK	1	Event ID
<code>session_id</code>	VARCHAR	<code>session_1776435900666_syzu2gq6lrk</code>	Browser session ID
<code>role</code>	VARCHAR	patient	User role (patient/physician)
<code>visit_type</code>	VARCHAR	first	Visit type (first/followup)
<code>file</code>	VARCHAR	<code>Input_TurboScribe SID 33.csv</code>	Patient being viewed
<code>speaker</code>	VARCHAR	<code>Patient_Input_TurboScribe SID 33</code>	Patient identifier
<code>event_type</code>	VARCHAR	<code>cursor_proximity_leave</code>	Event type
<code>element_id</code>	VARCHAR	/	Target UI element
<code>event_data</code>	JSONB	<code>{cursorX: 0, hoverDuration: 38007071, ...}</code>	Event details
<code>device_type</code>	VARCHAR	desktop	Device type
<code>client_timestamp</code>	TIMESTAMP	14:25:00	Client-side time
<code>created_at</code>	TIMESTAMP	14:25:00	Server save time

Used by: - `POST /api/tracking/events` —event batch submission from frontend - `GET /api/tracking/stats` —admin tracking dashboard - `GET /api/tracking/analytics` —behavior analysis

9. session_recording

Session replay data —stores rrweb-recorded user sessions as binary chunks, enabling video-like playback of how users navigated the dashboard. PHI (Protected Health Information) is masked so patient names and sentence text appear as “*” in recordings.**

While `user_interaction_log` records individual events, this table stores the raw data needed to replay the entire session visually. Chunks are sent every 30 seconds or every 500 events.

Where this appears in the UI: In the **AdminTrackingDashboard** (`AdminTrackingDashboard.tsx`), the “Recordings” tab lists all recorded sessions. Clicking a session opens the **rrweb-player** component, which plays back the session as a video showing mouse movements, clicks, scrolls, and page content (with PHI masked as “*”). Not visible to patients or doctors.

Column	Type	Sample	Description
id	SERIAL PK	1	Recording ID
session_id	VARCHAR	rec_1776397890521_hp3r5x	Recording session ID
chunk_index	INT	0	Chunk number (split every 30s or 500 events)
file	VARCHAR	Input_TurboScribe SID 33.csv	Patient being viewed
visit_type	VARCHAR	first	Visit type
recording_data	BYTEA	(87 bytes)	rrweb event data (PHI masked)
event_count	INT	2	Events in this chunk
created_at	TIMESTAMP	14:25:30	Server save time

Used by: - POST /api/tracking/recordings —chunk upload from frontend (sessionRecorder.ts) - GET /api/tracking/recordings —recording list on admin dashboard - GET /api/tracking/recordings/{sessionId} —session replay playback

10. auth_user

Application user accounts —represents every individual that can authenticate against the backend (admin, regular user, read-only). Defined in `auth/models.py` and created automatically via `init_db.py` (Base.metadata).

Backing model for FastAPI's `get_current_user` dependency. The `role` column drives authorization checks across the doctor/admin endpoints. Stored credentials are bcrypt hashes; rows authored by external SSO will have `password_hash = NULL` and a non-local `auth_provider`.

Column	Type	Sample	Description
id	SERIAL PK	1	User identifier
username	VARCHAR	clinician_a	Login name (indexed)
email	VARCHAR UNIQUE	a@cedars-sinai.org	Email (unique constraint)
password_hash	VARCHAR	\$2b12...	bcrypt hash; NULL when SSO is used
role	VARCHAR	user	admin / user / readonly (CHECK constraint)
is_superuser	BOOLEAN	false	Bypass per-patient ACL when true
is_active	BOOLEAN	true	Soft-delete flag
auth_provider	VARCHAR	local	local / azure_ad / etc.
created_at	TIMESTAMP	2026-03-12T...	Account creation time
updated_at	TIMESTAMP	2026-03-15T...	Last update (auto-updated)

Used by: - `auth/__init__.py:21 get_current_user()` —FastAPI dependency on every protected route (delegates to the backend selected by `AUTH_MODE`)
- `auth/access_control.check_patient_access()` —called from `routes_doctor.py:50` and the patient-scoped endpoints in `routes_transcript.py` -
`auth/admin_routes.py (/api/auth/users GET/POST/PATCH/DELETE)` —implemented user-management endpoints (UI: in progress)

11. auth_api_key

Per-user API keys for the multi_key auth mode —lets one user own multiple revocable keys (e.g. dashboard token + CLI token). Defined in auth/models.py.

In multi_key mode, the backend reads X-API-Key from request headers, hashes it, and looks up key_hash in this table. The matching user_id becomes the request's current_user. is_active=false or expires_at < NOW() causes lookup failure → 401.

Column	Type	Sample	Description
id	SERIAL PK	1	Key record id
user_id	INT FK	1	→ auth_user.id (CASCADE on delete)
key_hash	VARCHAR	sha256(...)	Hashed API key (raw key never stored)
label	VARCHAR	dashboard-prod	Human-readable name
is_active	BOOLEAN	true	Revocation flag
created_at	TIMESTAMP	2026-03-12T...	Issue time
expires_at	TIMESTAMP	2027-03-12T...	Expiry (NULL = never)
last_used_at	TIMESTAMP	2026-04-17T...	Updated on every successful auth

Used by: - auth/backends/api_key.py and auth/backends/multi_key.py —request-time X-API-Key verification - auth/admin_routes.py POST /api/auth/users/{user_id}/keys (issue) and DELETE /api/auth/users/{user_id}/keys/{key_id} (revoke) —implemented; admin UI in progress

12. patient_access

Mapping of which auth_users may access which patient files —the per-patient ACL layer on top of the role-based check.

A user may have a role of user (default) but still need an explicit grant per patient before any doctor endpoint will return data for that patient. is_superuser=true bypasses this check entirely. Composite unique constraint (user_id, patient_id) prevents duplicate grants.

Column	Type	Sample	Description
id	SERIAL PK	1	Grant id
user_id	INT FK	1	→ auth_user.id (CASCADE on delete)
patient_id	VARCHAR	SID_10	Patient identifier (matches transcript_analysis_log.patient_id)
access_type	VARCHAR	read	read / write / admin (CHECK constraint)
granted_at	TIMESTAMP	2026-04-01T...	When the grant was created
granted_by	INT FK	1	→ auth_user.id of the granter (NULL allowed)

Used by: - auth/access_control.check_patient_access(patient_id, user, db) —called at the top of /sentences/{file}/{speaker} (routes_doctor.py:50) and all patient-scoped transcript endpoints (routes_transcript.py) - auth/admin_routes.py POST /api/auth/users/{user_id}/patients (grant) and DELETE /api/auth/users/{user_id}/patients/{patient_id} (revoke) —implemented; admin UI in progress